

Relatório de Auditoria de Segurança

Pulso — Controle Financeiro

Versão auditada: **3.1.0**

Data: 29/08/2026

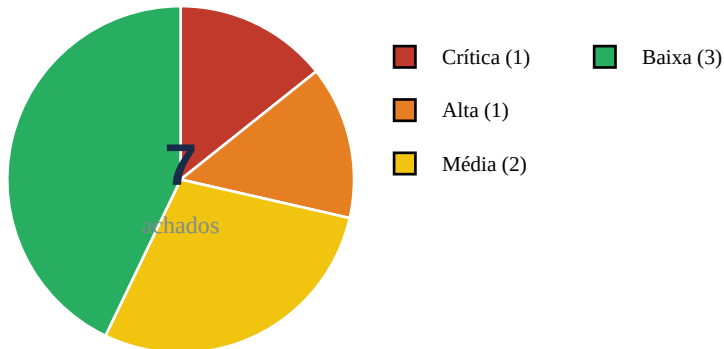
Escopo: 5 categorias (isolamento de tenant, permissão no navegador, IDOR, chaves expostas, XSS)

Stack: Frontend vanilla HTML/CSS/JS (sem framework) · Firebase Auth (email/senha + Google) · Firebase Realtime Database (RTDB) · Cloudflare Pages (deploy por push GitHub) · Capacitor Android · Sem backend próprio (regras de negócio no cliente + Security Rules)

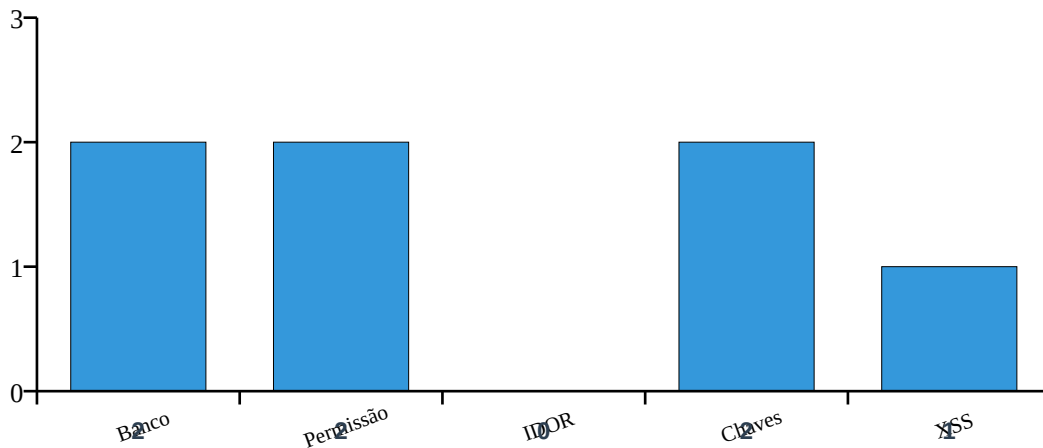
1. Resumo Executivo

O aplicativo Pulso é uma SPA estática (vanilla JS) cuja única camada de backend é o Firebase (Auth + Realtime Database). Não há servidor próprio, logo a segurança depende integralmente das **Firestore Security Rules** e do tratamento de dados no cliente. A auditoria cobriu 5 categorias e identificou **7 achados**.

Ponto forte: O isolamento por usuário (users/\$uid com auth.uid === \$uid) está correto e todas as operações de dado usam escapeHTML de forma consistente, ausência de eval/new Function. **Ponto fraco:** existe um path RTDB ('investimentos') público sem autenticação, as Security Rules não cobrem todos os nós, e a CSP permite unsafe-inline/unsafe-eval.



Distribuição por categoria (total de achados por tema):



2. Pontos Fortes e Fracos

Pontos Fortes

- Isolamento de usuário por UID em todas as operações RTDB (users/\$uid).
- Ausência de eval(), new Function(), v-html ou dangerouslySetInnerHTML.
- Uso consistente de escapeHTML() em ~30 locais de innerHTML com dado do usuário.
- Auth gate via onAuthStateChanged redireciona para login.html quando não autenticado.
- Upload de arquivos (avatar/Excel) usa FileReader sem execução de código.

Pontos Fracos

- Path 'investimentos' do RTDB legível/escutável sem auth (default público).
- Security Rules não cobrem todos os paths (ex.: sharedWith, investimentos).
- Config Firebase (apiKey, projectId, DB URL) hardcoded e commitado no git.
- CSP com 'unsafe-inline' e 'unsafe-eval' enfraquece defesa contra XSS.
- Service Worker e _headers (CSS immutable 1 ano) podem servir versão vulnerável.
- Dependência de regex customizada para escape, sem DOMPurify (defesa em profundidade).

3. Tabela de Achados

ID	Categoria	Severidade	Prioridade
F-01	Banco sem tranca	CRÍTICA	P0
F-02	Chaves expostas	ALTA	P1
F-03	Permissão definida no navegador	MÉDIA	P2
F-04	Banco sem tranca	MÉDIA	P1
F-05	Chaves expostas	BAIXA	P2
F-06	Inputs sem tratamento	BAIXA	P2
F-07	Permissão definida no navegador	BAIXA	P3

4. Detalhamento dos Achados

F-01 — Path 'investimentos' do RTDB acessível sem autenticação

CRÍTICA

Categoria: Banco sem tranca (Isolamento de tenant) | Prioridade: P0
Localização: script.js:2655, script.js:2658, script.js:2769

Descrição

O banco de dados Firebase Realtime Database (RTDB) expõe um path raiz 'investimentos' que é lido e escutado sem qualquer verificação de auth. Em database-rules.json não há regra para esse path, logo o comportamento default do Firebase é 'read/write público'. O código faz `fetch(INVEST_DB_URL + 'investimentos.json')` e `db.ref('investimentos')` sem token. Qualquer pessoa com a URL do projeto pode ler (e possivelmente escrever) dados de investimento de todos os usuários.

Recomendação de correção

Adicionar em database-rules.json uma regra restritiva para 'investimentos' (ex.: `read/write: false`, ou restringir por usuário convidado). Se o dado for realmente público (cotações), mover para um path dedicado e documentar; nunca misturar dado público com dado de usuário no mesmo nó raiz.

F-02 — Configuração Firebase (apiKey, projectId, DB URL) hardcoded e commitada

ALTA

Categoria: Chaves expostas | **Prioridade:** P1

Localização: firebase-config.js:5-11, www/firebase-config.js:5-11, script.js:7-15, android/app/google-services.json

Descrição

O arquivo firebase-config.js contém um comentário '■■■ NUNCA faça commit deste arquivo! (.gitignore deve protegê-lo)', porém o arquivo está commitado no repositório (raiz e www/), e o mesmo config foi duplicado dentro de script.js. A apiKey do Firebase é pública por design (client SDK), mas o vazamento de projectId, authDomain, databaseURL e storageBucket expõe a superfície de ataque: o projeto pode sofrer abuso de quota e tentativas diretas de acesso ao RTDB (agravado pelo item F-01). Há também o google-services.json do app Android com as mesmas chaves.

Recomendação de correção

Embora a apiKey deva existir no cliente, o ideal é: (1) garantir .gitignore cobrindo firebase-config.js e google-services.json; (2) renovar a apiKey no console Firebase e invalidar a vazada; (3) restringir o projeto por Application Restriction (HTTP referrer da sua origem + SHA-1 do app Android) na API Key; (4) nunca versionar google-services.json.

F-03 — Content-Security-Policy permite 'unsafe-inline' e 'unsafe-eval'

MÉDIA

Categoria: Permissão definida no navegador (CSP) | **Prioridade:** P2

Localização: index.html:9, login.html:9

Descrição

Ambas as páginas definem CSP com script-src permitindo 'unsafe-inline' e 'unsafe-eval'. Isso anula boa parte da proteção contra XSS: um vetor de injeção de script (mesmo com escapeHTML ausente em algum ponto futuro) executaria livremente. O app usa muitos <script> inline e eval indireto via Template literals, justificando o relaxed CSP, mas isso eleva o risco.

Recomendação de correção

Migrar scripts para arquivos externos com nonce, remover 'unsafe-inline' e 'unsafe-eval'. Para o Firebase SDK (que usa eval internamente em alguns modos), utilizar a build de produção (firebase-app-compat.js) ou mover para módulos ESM. Aplicar pelo menos 'default-src https:'. Validar via cabeçalho HTTP (Cloudflare _headers) em vez de meta tag.

F-04 — Regras RTDB não cobrem todos os paths de dados do usuário

MÉDIA

Categoria: Banco sem tranca (Isolamento de tenant) | **Prioridade:** P1
Localização: database-rules.json:1-15

Descrição

database-rules.json protege apenas users/\$uid com auth.uid === \$uid (correto para isolamento). Porém paths auxiliares como 'investimentos' (F-01) e possíveis nós de compartilhamento ('sharedWith') não têm regra explícita, ficando com default público. Além disso, não há validação de schema/tipo ('.write' sem restrição de estrutura), permitindo que um usuário escreva estruturas inesperadas que quebrem a app ou exfiltrem dados.

Recomendação de correção

Cobrir TODOS os paths em database-rules.json com regras explícitas (deny por default). Adicionar validação de tipo com o operador 'newData.hasChildren()' e '.validate' para campos obrigatórios. Revisar o nó 'sharedWith' para garantir que só o proprietário pode conceder acesso.

F-05 — Service Worker pode servir conteúdo stale/cacheado de versões antigas

BAIXA

Categoria: Chaves expostas | **Prioridade:** P2
Localização: service-worker.js (registrado em script.js), _headers:33

Descrição

O service-worker.js é registrado mas não foi auditado em detalhe. Combinado com _headers que define *.css como 'immutable' por 1 ano (exigiu cache-bust manual ?v=3.1.0), há risco de o SW servir uma versão antiga do app com código vulnerável após correções. Se o SW interceptar fetch de dados sensíveis, pode vaziar em cache.

Recomendação de correção

Auditar o service-worker.js: garantir que NÃO cacheia responses de RTDB/Firebase Auth (network-only para dados). Implementar estratégia de cache-bust por versão no nome do SW. Remover 'immutable' de _headers para CSS e usar cache-control com revalidação.

F-06 — Uso consistente de escapeHTML, mas dependência de regex customizada

BAIXA

Categoria: Inputs sem tratamento (XSS) | **Prioridade:** P2

Localização: script.js:41-45, usado em ~30 locais de innerHTML

Descrição

O app utiliza escapeHTML() (substitui & < > " ') em TODOS os innerHTML com dados do usuário (transações, contas, cartões, metas, ativos, etc.). Não foram encontrados eval(), new Function(), nem v-html/dangerouslySetInnerHTML. No entanto, a função é uma regex simples; se algum novo dev esquecer de chamá-la em um innerHTML futuro, haverá XSS. Não há defesa em profundidade (ex.: DOMPurify).

Recomendação de correção

Manter escapeHTML como padrão, mas adicionar DOMPurify como camada extra em toda inserção de HTML. Criar um helper seguro 'setHTML(el, str)' que sempre sanitiza. Adicionar lint rule (eslint-plugin-no-unsanitized) para bloquear innerHTML sem sanitização no CI.

F-07 — auth.onAuthStateChanged usado para gate, mas sem proteção de rota server-side

BAIXA

Categoria: Permissão definida no navegador (Auth state) | **Prioridade:** P3

Localização: script.js:51-56, login.html, index.html

Descrição

A proteção de página depende exclusivamente de onAuthStateChanged no cliente: se não houver usuário, redireciona para login.html. Isso é correto para uma SPA estática (não há servidor para proteger rotas), mas significa que qualquer usuário pode abrir index.html e ver o shell da app antes do redirect. O dado só é carregado após auth, então não há vazamento direto, mas o design confia 100% nas Security Rules do Firebase (que devem ser perfeitas — ver F-01/F-04).

Recomendação de correção

Manter arquitetura client-side, mas garantir que as Security Rules sejam a única fonte de verdade (testadas com firestore/rtdb emulator). Documentar que o 'gate' de UI é apenas UX, não segurança. Considerar mover para um bundle ofuscado (esbuild/terser) para dificultar engenharia reversa das regras de negócio.

5. Plano de Remediação Priorizado

Prio	Prazo	Achados	Ação
P0	Imediato	F-01	Corrigir regras RTDB do path 'investimentos' (deny por default ou restringir por usuário).
P1	Esta semana	F-02, F-04	Renovar apiKey Firebase, aplicar Application Restriction, adicionar .gitignore para configs, co
P2	Este mês	F-03, F-05, F-06	Endurecer CSP (nonce, sem unsafe-inline/eval), auditar service-worker, adicionar DOMPurify
P3	Backlog	F-07	Documentar que gate de UI ≠ segurança; ofuscar bundle; testar Security Rules no emulator.

6. Issues GitHub Sugeridas (Markdown)

Copie o bloco abaixo para criar as issues no repositório:

```
## F-01 [CRÍTICA] RTDB 'investimentos' público sem auth
- Arquivos: script.js:2655, script.js:2658, script.js:2769
- Ação: adicionar regra em database-rules.json negando read/write por default ou restringindo por usuário.
- Prioridade: P0

## F-02 [ALTA] Config Firebase hardcoded e commitado
- Arquivos: firebase-config.js:5-11, www/firebase-config.js:5-11, script.js:7-15, android/app/google-services.json
- Ação: .gitignore para configs, renovar apiKey, Application Restriction por HTTP referrer + SHA-1.
- Prioridade: P1

## F-03 [MÉDIA] CSP com unsafe-inline/unsafe-eval
- Arquivos: index.html:9, login.html:9
- Ação: nonce em scripts, remover unsafe-inline/eval, aplicar via _headers.
- Prioridade: P2

## F-04 [MÉDIA] Security Rules não cobrem todos os paths
- Arquivos: database-rules.json:1-15
- Ação: regras explícitas para todos os nós + validação de schema (.validate).
- Prioridade: P1

## F-05 [BAIXA] Service Worker / _headers CSS immutable
- Arquivos: service-worker.js, _headers:33
- Ação: não cachear respostas de RTDB/Auth; cache-bust por versão no SW.
- Prioridade: P2

## F-06 [BAIXA] escapeHTML sem defesa em profundidade
- Arquivos: script.js:41-45 (~30 usos)
- Ação: adicionar DOMPurify + eslint-plugin-no-unsanitized.
- Prioridade: P2

## F-07 [BAIXA] Gate de auth client-side apenas
- Arquivos: script.js:51-56
- Ação: documentar que UI gate ≠ segurança; testar Rules no emulator.
- Prioridade: P3
```